Universidad de los Andes

Ingeniería de Sistemas y Computación

ISIS-1611: Inteligencia Artificial

Semestre 2026-2

Taller 3: Razonamiento bajo incertidumbre

Isabela Archbold -202420166

Isabella Largacha - 2024020593

Samuel Zuluaga -202520012

Juan José Castro Maldonado -202520030

Juan David Forero - 202411707

13 de octubre de 2026

Punto 1

a) Tabla conjunta P(Incendio, Alarma) por la regla del producto

Incendio	Alarma	Cálculo P(I) · P(A|I)	Probabilidad
0	0	0.855000	No
0	1	0.045000	Sí
1	0	0.010000	No
1	1	0.090000	Sí

Para calcular la probabilidad conjunta de Incendio y Alarma, se utiliza la regla del producto P(Incendio, Alarma) = P(Incendio) × P(Alarma | Incendio).

- P(I=0, A=0) = 0.90 × 0.95 = 0.855
- P(I=0, A=1) = 0.90 × 0.05 = 0.045
- P(I=1, A=0) = 0.10 × 0.10 = 0.010
- P(I=1, A=1) = 0.10 × 0.90 = 0.090

Suma: 0.855 + 0.045 + 0.010 + 0.090 = 1.

- P(Alarma) = 0.045 + 0.090 = 0.135
- P(Incendio | Alarma) = 0.090 / 0.135 = 0.6667

Por lo tanto, si suena la alarma, la probabilidad de que haya un incendio es del 66.67 %.

b) Normalización

La función normalize convierte un diccionario de pesos en una distribución de probabilidades. Primero, verifica que el diccionario no esté vacío, que todos los pesos sean finitos y no negativos, y que su suma sea diferente de cero. Si se cumplen estas condiciones, se calcula la suma total de los pesos y se divide cada uno entre ese valor. Los resultados se guardan en un nuevo diccionario, manteniendo los valores originales del inicial. Finalmente, se verifica que la suma de las probabilidades obtenidas sea igual a 1.

c) Utilidad esperada y mejor acción

Acción	Cálculo EU(a) = sum P(s) U(a,s)	EU(a)
monitorear	0.9(0) + 0.1(-100)	-10.00
inspeccionar	0.9(-8) + 0.1(-25)	-9.70

brigada	0.9(-30) + 0.1(-5)	-27.50

Mejor acción: -9.70

d) Resultados de los tres casos

Caso	Evidencia	P(Incendio)	Acción elegida	EU de la acción
sin_evidencia	ninguna	0.100000	inspeccionar	-9.700000
alarma	Alarma=1	0.666667	brigada	-13.333333
sin_alarma	Alarma=0	0.011561	monitorear	-1.156069

Comparación de acciones:
Sin evidencia, la probabilidad de incendio es del 10% y la mejor decisión es inspeccionar. Cuando se activa la alarma, esta probabilidad aumenta al 66.67%, por lo que enviar una brigada se convierte en la acción con mayor utilidad esperada. En cambio, cuando no suena la alarma, la probabilidad disminuye al 1.16% y la mejor opción es monitorear. Esto muestra cómo la evidencia cambia las probabilidades y, con ellas, la decisión que ofrece mayor utilidad esperada.

Punto 2

a) Explicación de enumeration_ask

Enumeration_ask calcula la Probabilidad dada la consulta y la evidencia, P(consulta | evidencia), haciendo un recorrido por las variables de la red dependiendo de su orden topológico. Para esto, recorre secuencialmente la lista completa de variables en orden topológico mediante una función recursiva auxiliar (enumerate_all), que en cada paso procesa la primera variable de la lista y continúa con el resto. Para cada uno de los dos valores posibles en la consulta (0 y 1), se construye una copia de la evidencia que fija también el valor de la consulta, y se calcula probabilidad conjunta del evento completo. La variable fijada aporta su probabilidad puntual, mientras que cada variable oculta se suma sobre sus dos posibles. Por el mismo orden topológico, al llegar a una variable ya se conocen los valores de sus padres, los cuales sirven para consultar su tabla de probabilidad condicional. Ambos pesos resultantes se normalizan para poder obtener la distribución completa. Si en la evidencia ya se encuentra la variable de consulta, pues la rama contraria al valor fijado queda en peso 0, y si la evidencia es imposible (ambos en pesos 0), la normalización lanza ValueError. Para todo este proceso se usan copias del diccionario de evidencia para no cambiar el original.

b) Verificación en sensor_basico

Método	P(Incendio=1 | Alarma=1)
Cálculo manual	Reutilizando el cálculo del Punto 1 (regla de Bayes): P(Incendio=1, Alarma=1) = 0.10 × 0.90 = 0.090 P(Incendio=0, Alarma=1) = 0.90 × 0.05 = 0.045 P(Alarma=1) = 0.090 + 0.045 = 0.135 P(Incendio=1 | Alarma=1) = 0.090 / 0.135 = 0.666667
enumeration_ask	0.666667
Diferencia	0. Tiene sentido porque la funcion hace la misma operación que el calculo manual (producto y regla de bayes). Al ser una red de sólo dos variables con pocas operaciones de punto flotante, no hay acumulación de error de redondeo.

**Enumeration_ask

```
PS C:\Users\User\Desktop\s.zuluaga\IA\ISIS1611-Taller3-G3\AlertaBosque> python main.py -p 2 -l sensor_basico --case alarma -t

Alerta Bosque | Punto 2 | Sensor básico
Consulta: Incendio | Evidencia: Alarma=1
Método: Enumeración
Resultado  | Probabilidad
-----------+-------------
Incendio=0 | 0.333333
Incendio=1 | 0.666667

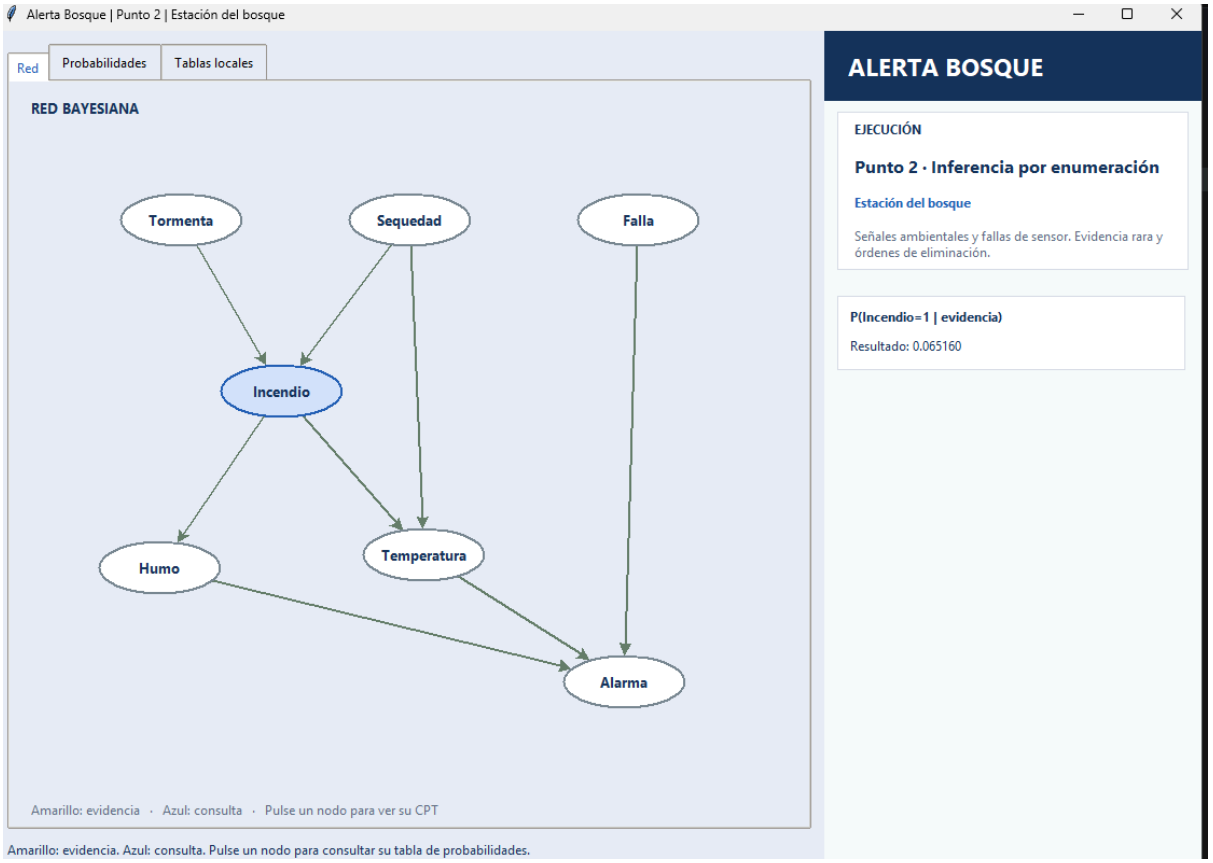
Estructura y probabilidades locales:

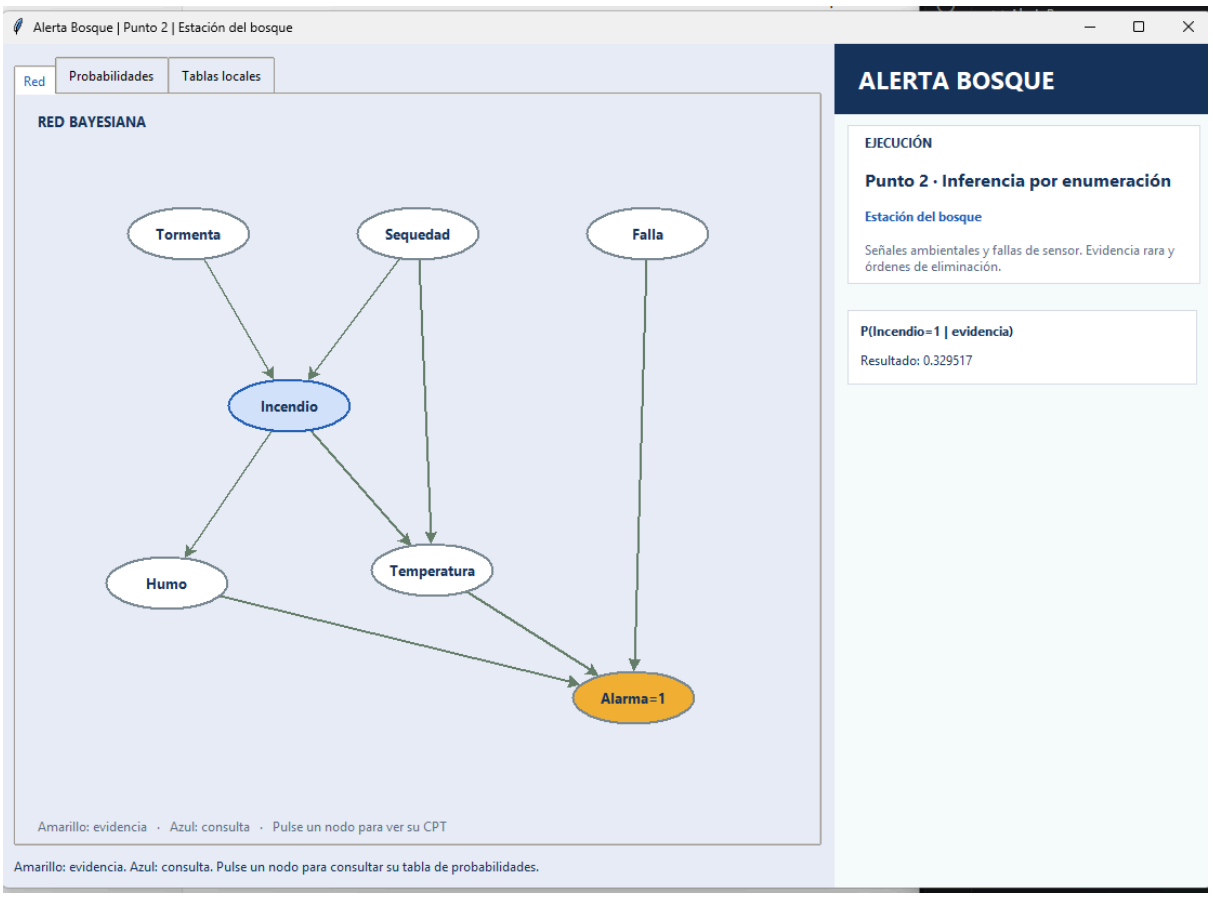
Incendio | Padres: ninguno
P(0)     | P(1)
---------+---------
0.900000 | 0.100000

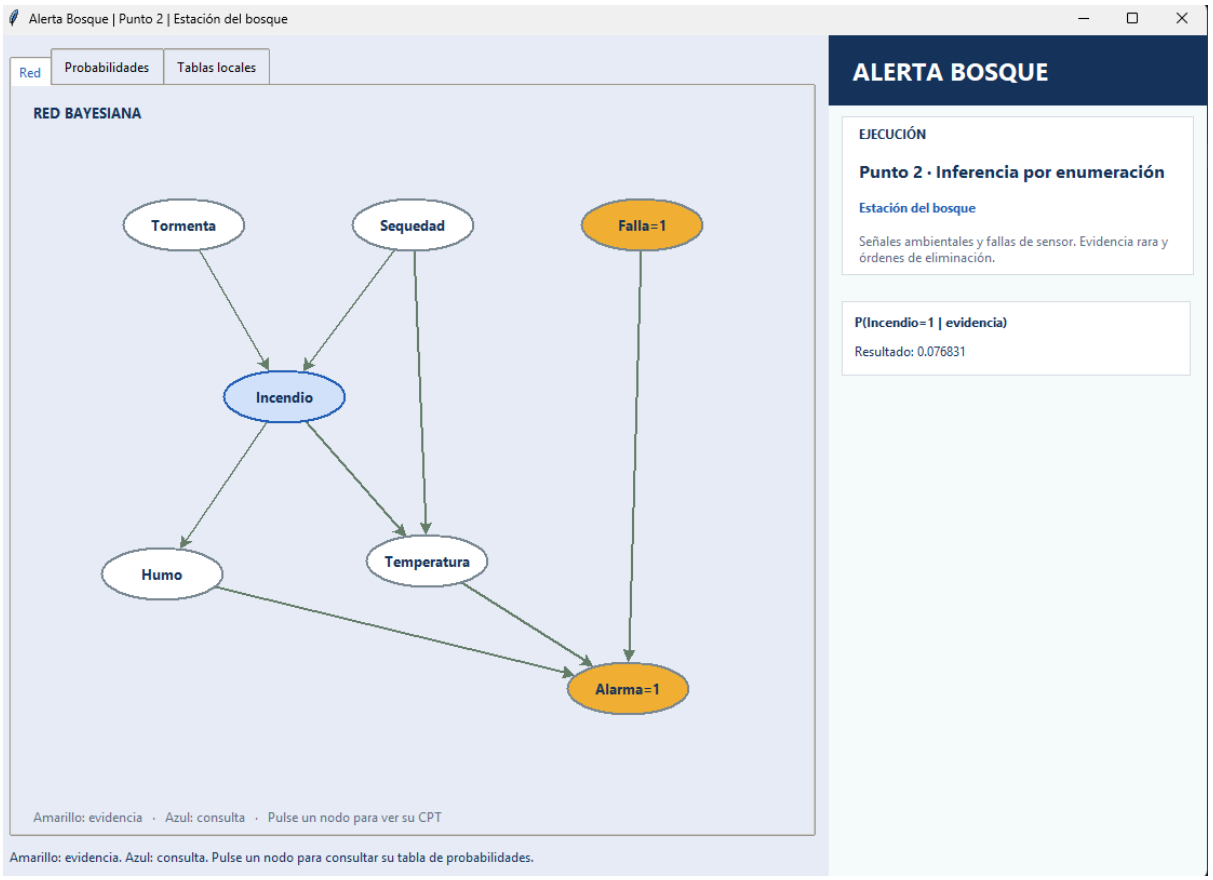
Alarma | Padres: Incendio
Incendio | P(0)     | P(1)
---------+----------+---------
0        | 0.950000 | 0.050000
1        | 0.100000 | 0.900000
PS C:\Users\User\Desktop\s.zuluaga\IA\ISIS1611-Taller3-G3\AlertaBosque>
```

Tabla de bosque: P(Incendio=1)

Caso	Evidencia	P(Incendio=1)	Descripción breve
sin_evidencia	ninguna	0.065160	Probabilidad inicial sin ninguna observación (ni falsa alarma, ni alarma)
alarma	Alarma=1	0.329517	Sube casi 5 veces más a la inicial, ya que la alarma es evidencia a favor del incendio.
alarma_falla	Alarma=1, Falla=1	0.076831	Baja casi al nivel de la base, ya que la falla del sensor explica la alarma sin necesidad de incendio real.


Alerta Bosque | Punto 2 | Estación del bosque
Red
Probabilidades
Tablas locales
RED BAYESIANA
Tormenta
Sequedad
Falla
Incendio
Humo
Temperatura
Alarma
Amarillo: evidencia · Azul: consulta · Pulse un nodo para ver su CPT
ALERTA BOSQUE
EJECUCIÓN
Punto 2 · Inferencia por enumeración
Estación del bosque
Señales ambientales y fallas de sensor. Evidencia rara y órdenes de eliminación.
P(Incendio=1 | evidencia)
Resultado: 0.065160
Amarillo: evidencia. Azul: consulta. Pulse un nodo para consultar su tabla de probabilidades.


Alerta Bosque | Punto 2 | Estación del bosque
Red
Probabilidades
Tablas locales
RED BAYESIANA
Tormenta
Sequedad
Falla
Incendio
Humo
Temperatura
Alarma=1
Amarillo: evidencia · Azul: consulta · Pulse un nodo para ver su CPT
Amarillo: evidencia. Azul: consulta. Pulse un nodo para consultar su tabla de probabilidades.
ALERTA BOSQUE
EJECUCIÓN
Punto 2 · Inferencia por enumeración
Estación del bosque
Señales ambientales y fallas de sensor. Evidencia rara y órdenes de eliminación.
P(Incendio=1 | evidencia)
Resultado: 0.329517


Alerta Bosque | Punto 2 | Estación del bosque
Red
Probabilidades
Tablas locales
RED BAYESIANA
Tormenta
Sequedad
Falla=1
Incendio
Humo
Temperatura
Alarma=1
Amarillo: evidencia · Azul: consulta · Pulse un nodo para ver su CPT
Amarillo: evidencia. Azul: consulta. Pulse un nodo para consultar su tabla de probabilidades.
ALERTA BOSQUE
EJECUCIÓN
Punto 2 · Inferencia por enumeración
Estación del bosque
Señales ambientales y fallas de sensor. Evidencia rara y órdenes de eliminación.
P(Incendio=1 | evidencia)
Resultado: 0.076831

Describir brevemente cómo cambia la probabilidad con cada evidencia.

c) cadena_humo: independencia condicional de Humo y Sequedad dado Incendio

Justificar con la estructura de la red (Sequedad → Incendio → Humo).

Condición	Resultado
P(Humo=1 | Incendio=1, Sequedad=0)	0.800000
P(Humo=1 | Incendio=1, Sequedad=1)	0.800000

```
Alerta Bosque | Punto 2 | Cadena de humo
Consulta: Humo | Evidencia: Incendio=1, Sequedad=0
Método: Enumeración
Resultado | Probabilidad
----------+-------------
Humo=0    | 0.200000
Humo=1    | 0.800000

Estructura y probabilidades locales:

Sequedad | Padres: ninguno
P(0)     | P(1)
---------+---------
0.600000 | 0.400000

Incendio | Padres: Sequedad
Sequedad | P(0)     | P(1)
---------+----------+---------
0        | 0.980000 | 0.020000
1        | 0.700000 | 0.300000

Humo | Padres: Incendio
Incendio | P(0)     | P(1)
---------+----------+---------
0        | 0.900000 | 0.100000
1        | 0.200000 | 0.800000
```

```
Alerta Bosque | Punto 2 | Cadena de humo
Consulta: Humo | Evidencia: Incendio=1, Sequedad=1
Método: Enumeración
Resultado | Probabilidad
----------+-------------
Humo=0    | 0.200000
Humo=1    | 0.800000

Estructura y probabilidades locales:

Sequedad | Padres: ninguno
P(0)     | P(1)
---------+---------
0.600000 | 0.400000

Incendio | Padres: Sequedad
Sequedad | P(0)     | P(1)
---------+----------+---------
0        | 0.980000 | 0.020000
1        | 0.700000 | 0.300000

Humo | Padres: Incendio
Incendio | P(0)     | P(1)
---------+----------+---------
0        | 0.900000 | 0.100000
1        | 0.200000 | 0.800000
PS C:\Users\User\Desktop\s.zuluaga\IA\ISIS1611-Taller3-G3\AlertaBosque>
```

¿Por qué la igualdad no implica independencia sin observar Incendio?

Siguiendo el orden, la estructura sería Sequedad ↠ Incendio ↠ Humo, siendo así una secuencia en cadena, con Incendio como nodo intermedio. Como ya sabemos, en redes bayesianas hay tres tipos de conexión; cadena, causa común y colisionador y el camino de cada una se activa o bloquea dependiendo si el nodo intermedio está observado. Para este caso, en cadenas, observar el nodo intermedio (Incendio) bloquea el camino, volviendo las dos variables extremas condicionalmente independientes.

Por ende, observar Incendio bloquea el camino entre Sequedad y Humo, dejándolas d-separadas dado Incendio, osea se puede asumir que : P(Humo | Incendio, Sequedad) = P(Humo |Incendio). Lo cual coincide con el resultado dado de P(Humo=1 | Incendio=1, Sequedad=0) = P(Humo=1 | Incendio=1, Sequedad=1) = 0.800000.

Sin embargo, esa regla también explica por qué la igualdad no implica independencia incondicional. Ya que si no observamos Incendio, el camino queda activo y la información si fluye, la Sequedad cambia la creencia sobre Incendio, y al mismo tiempo la de Humo. Acá la independencia condicional depende de que se observa, y no es una propiedad fija, hace falta observar específicamente el nodo intermedio para bloquear el camino.

Punto 3

a) Eliminación manual de P(Humo=1) en cadena_humo

Paso	Variable eliminada	Factores de entrada	Factor intermedio resultante
1	Sequedad	P(Sequedad): P(S=1)=0,4, P(S=0)=0,6. P(Incendio| Sequedad): P(I=1|S=0)=0,02. P(i=1|S=1)=0,3	f_1(Incendio) = Σ□ P(s)·P(Incendio|s). f_1(I=0) = 0.6·0.98 + 0.4·0.7 = 0.868. f_1(I=1) = 0.6·0.02 + 0.4·0.3 = 0.132
2	Incendio	f_1 (Incendio) y P(Humo|Incendio): P(H=1|I=0)=0,1. P(H=1|I=1)=0,8	f_2(Humo) = Σ_i f_1(i)·P(Humo|i). f_2(H=1) = 0.868·0.1 + 0.132·0.8 = 0.1924. f_2(H=0) = 0.868·0.9 + 0.132·0.2 = 0.8076
Final	-	f_2 (Humo)	P(Humo =1)= 0.1924 (0.8076 + 0.1924 = 1, así que no hay que normalizar porque no hay evidencia)

Resultado de P(Humo=1)	Valor
Manual (lápiz)	0,1924
Herramienta: eliminación, orden topológico	0,192400

Los factores intermedios de la herramienta (f_1 = 0.868 y 0.132; f_2 = 0.8076 y 0.1924) coinciden con los del cálculo manual. El mayor factor producto fue de 4 filas.

b) bosque, caso alarma: Incendio por tres métodos

Método	P(Incendio=1 | Alarma=1)	Filas del mayor factor intermedio
Enumeración	0,329517	no aplica
Eliminación, --order forward	0,329517	8
Eliminación, --order reverse	0,329517	32

Los tres métodos dan 0,3295 porque calculan la misma suma sobre la conjunta. La eliminación sólo reordena las sumas, saca de ellas los factores que no dependen de la variable que se suma, por lo tanto el resultado no cambia. Sin embargo, el orden de estas sumas si cambia el costo. En Forward se elimina primero Tormenta, que solo aparece dos veces. El producto involucra 3 variables, lo que representa 8 filas, y ninguno de los pasos supera 8. En Reverse se elimina la temperatura. El producto de este junta 5 variables, lo que representa 32 filas en total. En conclusión, el orden de la eliminación no cambia la respuesta pero puede cuadruplicar el tamaño del factor más grande.

c) prior_sample

Prior_sample(network, rgn) genera un mundo completo. Crea un diccionario vacío (sample) y recorre network.variables, que ya tiene el orden topológico, así que los padres de cada variable ya están en sample. Para cada variable X, se consulta network.probability(X, 1, sample). Posteriormente se llama una sola vez a rng.random() y asigna X=1 si P(X=1|Padres), y X=0 en el caso contrario. Solo se usó el generador recibido con una llamada por variable y no fijo.

d) sampling_ask

Sampling_ask repite exactamente “samples” intentos. Este parámetro cuenta intentos en vez de aceptaciones. En cada uno de estos, se genera una muestra con prior_sample. Si reject = True y algún valor de la muestra coincide con la evidencia, la descarta con continue. Si es compatible se suma 1 al conteo de valor de query y 1 al número de aceptadas. Sin rechazo, la validación inicial ya exige evidencia vacía, por lo que todas las muestras se aceptan y se estima una marginal. Al final, si no se llega a aceptar ninguna retorna (none,0). Esto no representa que la evidencia sea imposible, sino que ningún intento la cumplió. Si hubo aceptadas, entonces se normaliza los conteos aceptados con normalice y retorna (distribución, aceptadas).

e) Muestreo en bosque

Método	Evidencia	Intentos	Estimación	Exacto	Aceptadas	Error
prior	ninguna	1000	0,0560	0,0652	1000	0,0092
prior	ninguna	10000	0,0634	0,0652	10000	0,0018
rechazo	Alarma=1	1000	0,22775	0,3295	191	0,0520
rechazo	Alarma=1	10000	0,3156	0,3295	1841	0,0139
rechazo	Tormenta=1, Falla=1, Alarma=1	1000	sin estimación	0,2608	0	no aplica
rechazo	Tormenta=1, Falla=1, Alarma=1	10000	0,0909	0,2608	11	0,1699

Con muestreo a priori y sin evidencia, las estimación se acerca al valor exacto 0,0652: pasa de 0.0560 con 1000 intentos a 0.0634 con 10000, y el error baja de 0.0092 a 0.0018. Todas las muestras se aceptan, porque no se descarta ninguna.

Con rechazo y Alarma = 1, el estimado mejora con más intentos también (0.2775 con 1000 y 0.3156 con 10000, frente al exacto 0.3295, con errores de 0.0520 y 0.0139). Aunque solo se aceptan 191 y 1841 muestras, cerca del 18% de los intentos, lo cual coincide con =(Alarma=1)=0,181. Por esto, el error es mayor que en el caso a priori con los mismos intentos, ya que en ese caso se estima con menos datos.

Con la evidencia rara (Tormenta = 1, Falla=1, Alarma=1), la evidencia tiene probabilidad de solo 0.0014. Con 1000 intentos no se aceptó ninguna muestra y no hay estimación. Esto último no significa que la evidencia sea imposible, solo o'que ningún intento la cumplió. Con 10000 intentos se aceptaron 11 muestras y estimado de 0,0909 con error de 0,1699 respecto al exacto con valor de 0,2608. Aumentar los intentos ayudó pero en casos con evidencia improbable, el rechazo desperdicia casi todo el trabajo. Ya que el muestreo es aleatorio, estos resultados son de una semilla (7), y otra semilla puede dar un error distinto a la actual.

Punto 4

a) Elementos del MDP en cuadrícula

Elemento	Descripción

Estados	Las 11 posiciones transitables (x, y) de la cuadrícula, incluidos los estados terminales. El origen (0, 0) está arriba a la izquierda. El muro en (1, 1) no pertenece al conjunto de estados y la posición inicial es (0, 2).
Acciones	En cada estado no terminal el dron puede elegir entre cuatro acciones: - norte (N) - este (E) - sur (S) - oeste (W) Los estados terminales no tienen acciones disponibles.
Transiciones	Con ruido de 0.20 el dron se mueve en la dirección elegida con probabilidad 0.80 y hacia cada dirección perpendicular con probabilidad 0.10. Si el movimiento lo lleva contra un muro o fuera del mapa, permanece en su posición. Las probabilidades que conducen al mismo estado sucesor se suman
Recompensas	Cada transición hacia un estado no terminal recibe una recompensa de -0.04, incluso si el dron rebota y permanece en su posición. Al entrar en G recibe +1 y al entrar en X recibe -1 en lugar de la recompensa por movimiento y la recompensa terminal se recibe una sola vez al entrar
Estados terminales	Los estados terminales son G, ubicado en (3, 0), y X, ubicado en (3, 1). Al llegar a cualquiera de ellos la misión termina. Ambos mantienen V(s) = 0 porque su recompensa ya se recibió al entrar y no hay recompensas futuras.
Descuento (gamma)	El factor de descuento es $\gamma = 0.90$. Una recompensa recibida un paso más tarde tiene un peso de 0.90 respecto a recibirla inmediatamente dos pasos más tarde su peso es 0.90^2. las recompensas futuras tienen menor peso cuanto más se demora en obtenerlas.

¿Por qué posición y acción bastan?

El modelo cumple con la propiedad de Markov porque, con el mapa y sus parámetros fijos, basta con conocer la posición actual del dron y la acción elegida para determinar las probabilidades del siguiente estado. Estas probabilidades dependen del ruido, los muros y los bordes cercanos, sin necesidad de conocer el recorrido anterior del dron.

b) action_value

La función action_value calcula el valor esperado de realizar una acción desde un estado. Para esto, recorre los posibles estados siguientes y, en cada uno, suma la recompensa inmediata y el valor

futuro descontado. Luego, multiplica este resultado por la probabilidad de la transición y acumula los valores obtenidos.

Se utilizan las transiciones y recompensas del modelo, que ya tienen en cuenta el viento y los rebotes contra muros y bordes. Además, los valores de los estados terminales se mantienen en cero para evitar contar dos veces la recompensa que se recibe al entrar en ellos.

```
Implement with Codex
# TODO: Add your code here
total = 0.0
for successor, probability in mdp.transitions(state, action):
    reward = mdp.reward(state, action, successor)
    total += probability * (reward + mdp.discount * values[successor])
return total
# FIN DEL BLOQUE DEL ESTUDIANTE
```

c) value_iteration

La iteración de valores comienza con Vo (s) = o para todos los estados. En cada barrido calcula el valor de cada estado no terminal como el máximo de action_value entre sus acciones disponibles es importante decir que los estados terminales mantienen su valor en cero. El barrido es sincrónico o sea que todos los cálculos consultan el diccionario values del barrido anterior y guardan sus resultados en new_values, después de recorrer todos los estados se reemplaza values por new_values

Por último, la variable delta registra el mayor cambio absoluto entre ambos barridos: δ = maxs |Vnuevo(s) − Vanterior(s)|. Se establece converged cuando delta < tolerance. La ejecución termina al cumplir ese criterio o al alcanzar el máximo de barridos más esto de alcanzar el máximo no garantiza la convergencia

d) Chequeo con control y propagación en cuadrícula

Mapa	Barrido	delta	Observación
control	1	1	El valor del inicio pasa de 0 a 1
control	2	0	Los valores no cambian y hay convergencia
cuadrícula	1	0.792	El valor del inicio es -0.040000
cuadrícula	2	0.56304	El valor del inicio es

			-0.076000
cuadrícula	3	0.398909	El valor del inicio es -0.108400
cuadrícula	25	1.57695 x 10^-8	El cambio máximo supera la tolerancia
cuadrícula	26	6.91094 x 10 ^-9	El cambio máximo es menor que 10^-8 y hay convergencia

Barridos hasta converger en cuadrícula: 26

Punto 5

a) extract_policy

```
def extract_policy(mdp, values):

   policy = {}

   for state in mdp.states:

       best_action = None

       best_value = float('-inf')

       for action in mdp.actions(state):

           q = action_value(mdp, state, action, values)

           if q > best_value:

               best_value = q

               best_action = action

       policy[state] = best_action

   return policy
```

Para construir extract_policy, se reutiliza action_value para comparar las acciones según su valor Q(s, a). Esto permite tener en cuenta el viento, ya que una acción puede llevar a una buena casilla,

pero también existe el riesgo de que el dron se desvíe hacia una pérdida. Además, las acciones se evalúan en el orden de mdp.actions (N, E, S, W) para respetar el desempate.

El algoritmo recorre todos los estados, comenzando con la mejor acción en None y el mejor valor en menos infinito. Luego, calcula el Q de cada acción y, si encuentra uno mayor, actualiza la mejor acción y su valor. Al final, guarda la acción seleccionada en policy.

La comparación estricta (>) permite que, si dos acciones tienen el mismo valor, se conserve la primera. Por ejemplo, en el escenario de atajo con ruido 0, en (0, 0) se tiene Q(E) = Q(S) = 1.155, por lo que se elige E. Además, como los estados terminales no tienen acciones disponibles, su valor en policy queda en None, como pide el enunciado.

b) atajo: ruido 0 vs. ruido 0.3

Tabla 5.1. Efecto del ruido en la política de atajo

Estado (x,y)	Acción (ruido 0)	Acción (ruido 0.3)	V (ruido 0)	V (ruido 0.3)
(0,2) INICIO	E	N	1.367	0.711
(1,2)	E	N	1.481	0.820
(2,2)	E	N	1.601	0.959
(3,2)	E	N	1.727	1.118
(4,2)	E	N	1.860	1.302
(5,2)	E	N	2	1.555
(0,3)	N	W	1.258	0.319
(6,3)	N	E	2	1.382

Nota. Se muestran sólo los estados cuya acción cambia. Los 14 estados no terminales restantes conservan la misma acción con ambos niveles de ruido.

Cambio de decisión identificado y explicación: El cambio más importante ocurre en el estado inicial (0, 2). Sin ruido, el dron elige E y avanza directamente hacia la descarga, con V(inicio) = 1.367. Cuando el ruido aumenta a 0.3, cambia a N, al igual que en los estados de (0, 2) a (5, 2).

Esto ocurre por el riesgo del viento. La fila inicial está justo encima de cinco estados de pérdida. Sin ruido, avanzar al Este es seguro, pero con ruido 0.3 existe una probabilidad de 0.15 de que el viento desvíe al dron hacia el Sur y caiga en una pérdida. Por eso, prefiere rodear por arriba, aunque tenga que recorrer más casillas.

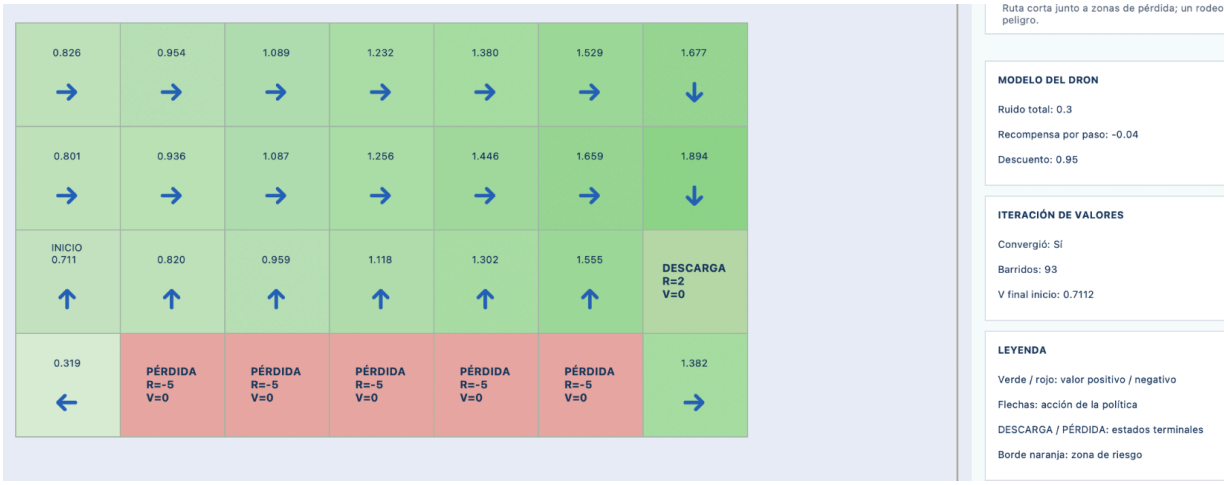
Algo similar ocurre en las esquinas inferiores. En (0, 3), la acción cambia de N a W y en (6, 3) cambia de N a E. Al elegir movimientos hacia los bordes, el dron reduce el riesgo de caer en estados peligrosos. Como resultado, V(inicio) baja de 1.367 a 0.711, porque el camino es más largo y el viento

hace que los movimientos sean menos seguros.

Figura 5.1. Evidencia atajo con ruido 0



Figura 5.2. Evidencia atajo con ruido 0.3



c) zona_riesgo: recompensa de vida y descuento

Tabla 5.2. Efecto de la recompensa por movimiento en zona_riesgo

Estado (x,y)	Acción (-0.04)	Acción (-0.30)	V (-0.04)	V (-0.30)
(0,1) INICIO	S	E	1.004	-0.716
(3,1) R	N	E	1.349	0.636

Nota. Se muestran sólo los estados cuya acción cambia. Los 18 estados restantes conservan la misma acción. Ambos casos usan ruido 0.1 y descuento 0.95; solo varía la recompensa. Ver Figuras 5.3 y 5.4.

Efecto de la recompensa por movimiento: Con recompensa -0.04, el dron prefiere evitar la zona de riesgo R y rodearla por arriba o por abajo. Cuando el costo por movimiento aumenta a -0.30, en (3, 1) la acción cambia de N a E, por lo que el dron prefiere atravesar la zona peligrosa en vez de rodearla.

Esto pasa porque atravesar requiere 3 pasos y una penalización adicional de 0.40, mientras que rodear necesita 5 pasos. Con costo -0.04, los dos pasos extra cuestan 0.08, por lo que es mejor rodear. En cambio, con -0.30, cuestan 0.60, así que conviene atravesar. Esto muestra que, cuando moverse cuesta más, el dron prefiere caminos cortos aunque tengan mayor riesgo. Además, V(inicio) baja de 1.004 a -0.716 por el aumento del costo de los movimientos.

Tabla 5.3. Efecto del descuento en zona_riesgo

Estado (x,y)	Acción (γ = 0.95)	Acción (γ = 0.50)	V (γ = 0.95)	V (γ = 0.50)
(0,1) INICIO	S	N	1.004	-0.073
(1,1)	N	W	1.089	-0.076
(2,1) R	N	S	1.207	-0.069

Nota. Se muestran sólo los estados cuya acción cambia.. Ambos casos usan ruido 0.1 y recompensa por movimiento -0.04; solo varía el descuento. Ver Figuras 5.3 y 5.5.

Efecto del descuento: Con descuento 0.95, el dron sigue dando importancia a la recompensa final y busca caminos hacia la descarga. Cuando el descuento baja a 0.50, las recompensas futuras pierden valor mucho más rápido. Por ejemplo, a cinco pasos de distancia, la recompensa de +2 vale menos de 0.07. Por eso, los estados más alejados de la meta tienen valores cercanos a cero, como V(inicio) = -0.073.

Este cambio también afecta las decisiones del dron. En (2, 1), la acción cambia de N a S para salir de la zona de riesgo, mientras que en (1, 1) cambia de N a W, alejándose de la meta. Esto muestra que el dron empieza a darle más importancia a los costos inmediatos que a llegar a la descarga. En cambio, cuando está cerca de la meta, sigue eligiendo acciones que lo acercan a ella porque la recompensa todavía tiene suficiente valor.

Figura 5.3. zona_riesgo, caso base (recompensa -0.04, descuento 0.95)



Figura 5.4. zona_riesgo con recompensa -0.30



Figura 5.5. zona_riesgo con descuento 0.5



Punto 6

6a)

I. ¿Cómo cambian la probabilidad de incendio y la decisión al recibir evidencia? Distinga probabilidad y utilidad esperada.

La evidencia afecta directamente la probabilidad de incendio y la decisión óptima. Sin evidencia, la probabilidad es del 10% y se elige inspeccionar. Cuando hay alarma, la probabilidad aumenta al 66,67% y la mejor acción es enviar una brigada. Sin alarma, la probabilidad disminuye al 1,16% y la mejor acción es monitorear. La probabilidad representa qué tan posible es que ocurra un incendio, mientras que la utilidad considera esa probabilidad y las consecuencias de cada acción. La acción más conveniente no depende solo del evento más probable, sino de cuál maximiza la utilidad esperada.

II. ¿Por qué enumeración y eliminación deben coincidir? ¿Qué efecto observó al cambiar el orden de eliminación?

III. ¿Cómo influyen los intentos y la probabilidad de la evidencia en el error y en la cantidad de muestras aceptadas?

Con mas intentos el error tiende a bajar. En prior, pasa de 0,0092 a 0,0018, en rechazo con alarma para de 0,0520 a 0,0139. Sin embargo, en una ejecución particular no esta garantizado. La probabilidad de la evidencia domina en las muestras aceptadas: P(Alarma = 1)= 0,181, asi que se acerca al 18% de los intentos. En el caro raro, P(Tormenta=1,Falla=1,Alarma=1)=0,0014, así que se esperan unas 14 muestras aceptadas de cada 10000 intentos. Con 1000 intentos no se aceptó ninguna y no hay una estimación. Con 10000 intentos se aceptaron 11 y el error fue de 0,17, porque el estimado se basa en muy pocos datos. Con 5 semillas distintas (7 a 11) y 10000 intentos, las aceptadas fueron 11,10,16,14,15, en donde el error varió de 0,011 a 0,170. Cuando más improbable es la evidencia, más intentos se desperdician y más ruidosos es el estimado.

IV. ¿Qué muestran los barridos de valores y qué criterio se usó para detenerlos?

Los barridos muestran cómo las recompensas se propagan por la cuadrícula mediante actualizaciones sucesivas cada barrido utiliza exclusivamente los valores del anterior. En el estado inicial los primeros tres valores fueron -0.04, -0.076 y -0.1084 ya que inicialmente predominan los costos de movimiento. Luego la recompensa de llegar a la meta influye en esa posición y su valor

converge a aproximadamente 0.373852 El criterio de parada fue que el mayor cambio absoluto entre dos barridos consecutivos fuera menor que la tolerancia de 10^-8 y en la cuadrícula se cumplió en el barrido 26, con un cambio máximo de 6.91094 x 10^-9 y también existía un límite de 1000 barridos que no fue necesario alcanzar

V. ¿Qué efecto observó al variar el ruido, la recompensa por movimiento y el descuento? Explique un cambio representativo y distinga una política de una ruta fija.

Al cambiar los parámetros del MDP, el dron toma decisiones diferentes según las condiciones. Cuando el ruido aumenta de 0 a 0.3, el dron busca caminos más seguros. Por ejemplo, en el estado inicial (0, 2), cambia de E a N para evitar que el viento lo lleve a una casilla de pérdida. Por esto, V(inicio) baja de 1.367 a 0.711 (Tabla 5.1).

Cuando el costo por movimiento pasa de -0.04 a -0.30, el dron prefiere caminos más cortos, aunque tengan más riesgo. En (3, 1), cambia de N a E porque rodear la zona peligrosa termina siendo más costoso que atravesarla (Tabla 5.2). Por otro lado, cuando el descuento baja de 0.95 a 0.50, al dron le importa menos la recompensa futura y se enfoca más en los costos inmediatos. Esto se ve en (2, 1), donde cambia de N a S para salir de la zona de riesgo, aunque no sea en dirección a la meta (Tabla 5.3).

Por último, una política no es lo mismo que una ruta fija. Una ruta indica los movimientos que se deben seguir en un orden específico, mientras que una política indica qué acción tomar desde cada estado. Esto permite que el dron cambie su decisión si el viento lo desvía. Por ejemplo, en atajo con ruido 0.3, la política asigna W a (0, 3) y E a (6, 3), casillas fuera del camino ideal a las que el viento puede llevar al dron (Tabla 5.1).

6b) Reflexión sobre el uso de IA

I. Juan José Castro Maldonado:

> En general para este taller entendí muy bien lo que debía hacer desde el inicio, por lo que no necesité utilizar inteligencia artificial para indicarme en que me debía enfocar y cómo hacer ciertas partes del código. Sin embargo, la IA me ayudó a recordar ciertas restricciones obligatorias para el funcionamiento de los algoritmos que no había implementado en mi código. Entre otras sugerencias menores de parte de la IA entra la sugerencia en la que me recomienda utilizar raise ValueError en vez de print, es una buena práctica.

II. Isabela Archbold Cardenas:

Este taller me pareció mucho más sencillo y dinámico que los anteriores, por lo que el uso de IA fue mínimo. Principalmente, la utilicé para mejorar la redacción de mis interpretaciones del punto 5 y para organizar mejor las tablas. Me sugirió mostrar únicamente las filas donde había cambios al comparar las condiciones, lo que permitió que el documento no quedara innecesariamente largo y

se enfocara en lo importante para el análisis.

También la utilicé para revisar mi código. En una primera versión, no estaba guardando la acción seleccionada en policy, por lo que retornaba un diccionario vacío. La IA me ayudó a identificar este error y yo hice la corrección. Después confirmé que el código funcionaba correctamente, lo que me dio más seguridad sobre mis resultados.

III. Samuel Zuluaga Robledo:
Este taller era mucho más corto que los otros, y los casos de la red bayesiana eran muy similares a los vistos en clase, por lo que el uso de IA no fue muy complejo. Además, hice mi parte después del parcial, así que tenía los temas frescos y necesite poca ayuda, la use principalmente para revisar el código de enumeration_ask (en especial para confirmar que el uso de .copy() en la recursión era necesario para no mutar la evidencia original) y no mucho más. El documento de análisis prácticamente no necesito IA, porque ya tenía clara la teoría detrás de la enumeración y la independencia condicional. Verifique mi solución comparando los resultados contra el cálculo manual de sensor básico y contra eliminación de variables, y sí podría explicar cada parte del código en una sustentación.

IV. Juan David Forero

En la realización de este taller, me apoyé mucho de los algoritmos en lenguaje natural y pseudocódigo que se encontraban en las diapositivas del curso. Asimismo, lo realice después de haber estudiado y presentado el parcial, por lo que los temas los tenía más estudiados que con los otros talleres. Generalmente mis errores se centran mucho en pequeñas correcciones a la hora de hacer operaciones o recursiones, pero gracias a la IA he podido mitigar esos errores que no necesariamente representan que no sepa como hacer el código. Adicionalmente, el tema de probabilidad que se maneja en este módulo, lo estoy viendo al mismo tiempo en otra materia, por lo que ha sido de gran ayuda poder reforzar esos temas mutuamente y desde distintas perspectivas..

Anexos: uso de inteligencia artificial

Función: normalize, expected_utility y best_action

Campo	Contenido
Versión inicial (sin IA)	`def normalize(weights: dict) -> dict:` `for peso in weights.values():`

```
        if peso < 0 or not
math.isfinite(peso):

            print("Peso inválido")



    total = sum(weights.values())



    if total == 0:

        print("La suma es cero")



    rta = {}



    for clave, peso in weights.items():

        rta[clave] = peso / total



    return rta
```

```
def expected_utility(distribution: dict,
utilities: dict) -> float:

if set(distribution) != set(utilities) or
not distribution:

        print("Probabilidades y utilidades
deben tener las mismas claves")

    if any(not math.isfinite(p) or p < 0 for
p in distribution.values()):

        print("Distribución inválida")

    if not
math.isclose(sum(distribution.values()), 1,
abs_tol=1e-9):

        print("La distribución debe sumar
```

```
uno")

    if any(not math.isfinite(u) for u in
utilities.values()):

        print("Utilidades no finitas")



    rta = 0



    for estado, probabilidad in
distribution.items():

        rta += probabilidad *
utilities[estado]



    return rta
```

```
def best_action(distribution: dict, actions:
dict) -> tuple[str, dict]:

if not actions:

        raise ValueError("Se requiere al
menos una acción")

    rta = {}

    mejor_accion = None

    mejor_utilidad = float("-inf")



    for accion, utilidades in
actions.items():

        utilidad =
expected_utility(distribution, utilidades)

        rta[accion] = utilidad
```

	``
	        if utilidad > mejor_utilidad:
	            mejor_utilidad = utilidad
	            mejor_accion = accion
	
	    return mejor_accion, rta
	``

Prompt exacto	"Por Favor revisa mi código, necesito que me digas que puedo mejorar. Te voy a adjuntar los requerimientos del mismo para que no se me pase nada".
Sugerencias relevantes	Incluir la condición que indica que el diccionario no puede estar vacío en normalize. Usar raise ValueError para las excepciones, esto ayudaría a que el código no se siguiera ejecutando pasadas las líneas de los prints.
Aceptado / rechazado / adaptado	Se aceptaron todos los cambios.
Cómo se verificó	Se determina si el código funciona ejecutando tres casos de prueba: sin evidencia, alarma y sin alarma, usando la terminal de VS Code (con `runner.py` y comandos por terminal). Se compararon las probabilidades y utilidades esperadas con los resultados teóricos calculados manualmente (como se muestra para el caso sin evidencia en la tabla bajo el subtítulo **Utilidad esperada y mejor acción**). Los tres casos dieron los resultados esperados.

Función: enumeration_ask (Punto 2)

Campo	Contenido

Versión inicial (sin IA)	```
	def enumeration_ask(network, query: str, evidence: dict) -> dict:
	network.validate_query(query, evidence)
	    pesos = {}
	``

	``
	    for valor in [0, 1]:
	        if query in evidence:
	            if evidence[query] != valor:
	                pesos[valor] = 0.0
	                continue
	            extended_evidence = evidence.copy()
	            extended_evidence[query] = valor
	
	            pesos[valor] = enumerate_all(network.variables, extended_evidence, network)
	
	def enumerate_all(variables, evidence, network):
	    """Calcula la probabilidad de un evento dado."""
	    if not variables:
	        return 1.0
	    variable = variables[0]
	    resto = variables[1:]
	    if variable in evidence:
	        return network.probability(variable, evidence[variable], evidence) * enumerate_all(resto, evidence, network)
	    else:
	        total = 0.0
	        for valor in (0, 1):
	            extended_evidence = evidence.copy()
	            extended_evidence[variable] = valor
	            total += network.probability(variable, valor, evidence) * enumerate_all(resto, extended_evidence, network)
	        return total
	``

Prompt exacto	“Tengo el esqueleto general de inferencia por enumeración de las notas de clase (Cap. 6.4, Listing 6.1) y la idea de

	factorizar con CPTs del Cap. 7.5.1, pero el libro no trae la recursión sobre las variables en orden topológico — la armé yo mismo. Te paso mi primer intento de enumeration_ask y enumerate_all , ¿cómo lo ves?" "El .copy() que hace en ambos lugares (enumeration_ask y enumerate_all), ¿es realmente necesario? ¿Cómo se vería un código que no use .copy() y falle por eso?"
Sugerencias relevantes	Primero señaló que en enumeration_ask faltaba el caso general: las líneas que construyen el evento extendido y calculan el peso estaban mal indentadas, dentro del if query in evidence, por lo que nunca se ejecutaban cuando la variable de consulta no estaba en la evidencia. Luego, para responder sobre .copy(), construyó una versión de prueba que mutaba directamente el diccionario de evidencia en vez de copiarlo, y la ejecutó sobre la red bosque: al reutilizar esa evidencia "contaminada" para una segunda consulta distinta (Tormenta, sin evidencia real), el resultado salió mal — 0.754 en vez de 0.9206, el valor correcto. Esto demostró en la práctica que .copy() es necesario para que cada rama de la recursión (y cada consulta) trabaje sobre su propio evento aislado, sin modificar el diccionario original.
Aceptado / rechazado / adaptado	Se aceptó la corrección de indentación en enumeration_ask. Se confirmó que el .copy() en enumerate_all, que ya estaba en mi primer intento, era obligatorio — no se cambió nada ahí, pero quedó claro el motivo con la prueba del resultado incorrecto.
Cómo se verificó	Usando todos los casos y poniendolos en codigo de la terminal. python main.py -p 2 -l sensor_basico --case alarma -t python main.py -p 2 -l bosque --case sin_evidencia -t python main.py -p 2 -l bosque --case alarma -t python main.py -p 2 -l bosque --case alarma_falla -t python main.py -p 2 -l cadena_humo --query Humo --evidence "Incendio=1,Sequedad=0" -t python main.py -p 2 -l cadena_humo --query Humo --evidence "Incendio=1,Sequedad=1" -t Se comparó el resultado de sensor_basico contra el cálculo manual del Punto 1 (2/3 = 0.666667, coincide). Los demás casos se verificaron comparando con la referencia exacta de eliminación de variables (bosque/alarma: ambos métodos dan 0.329517).

Función: prior_sample (Punto 3)

Campo	Contenido

Versión inicial (sin IA)	`def prior_sample(network, rng: random.Random) -> dict:`
	`    """Genera un mundo completo, muestreando padres antes que hijos."""`
	`    sample = {}`
	
	`    for variable in network.variables:`
	`        p_one = network.probability(variable, 1, sample)`
	`        sample[variable] = 1 if rng.random() > p_one else 0`
	
	`    return sample`

Prompt exacto	Estoy realizando una función dentro del contexto de este taller sobre inteligencia artificial y probabilidad, estoy teniendo dificultades porque los resultados de arroja una probabilidad inusual a la que yo deberia esperar

Sugerencias relevantes	“El error está en el operador de comparación: tu amigo usó `>` en lugar de `<`.
	
	python
	
	Python
	`sample[variable] = 1 if rng.random() > p_one else 0   # incorrecto`
	
	`sample[variable] = 1 if rng.random() < p_one else 0   # correcto`
	
	“
	
	se señalo que debido a un error en un signo, la probabilidad daba invertida, por lo que en vez de ser P, el resultado que

	arrojaba era 1-P
Aceptado / rechazado / adaptado	Se aceptó la corrección en el signo que estaba invertido, lo que dio un resultado acertado en el muestreo y las pruebas. Se recomendaron pequeños cambios en la estructura de la función pero estos no fueron considerados.
Cómo se verificó	Se comparó con el exacto: -p 3 -l bosque --case sin_evidencia -a prior -n 10000 , en donde el valor debía quedar cerca de 0,0652 con semilla 7, 0,0634

Función: sampling_ask (Punto 3)

Campo	Contenido

Versión inicial (sin IA)	`def sampling_ask(network, query, evidence, samples, rng, reject=False):`
	`  network.validate_query(query, evidence)`
	`  if samples < 1:`
	`  raise ValueError("El número de muestras debe ser positivo")`
	`  if evidence and not reject:`
	`  raise ValueError("Muestreo a priori estima marginales; use rechazo con evidencia")`
	
	`  counts = {0: 0, 1: 0}`
	`  accepted = 0`
	
	`  for _ in range(samples):`
	`  sample = prior_sample(network, rng)`
	
	`  if reject and any(sample[variable] != value for variable, value in evidence.items()):`
	`  continue`
	
	`  counts[sample[query]] += 1`
	`  accepted += 1`
	
	`  if accepted == 0:`
	`  return None, 0`

	`return normalize({0: counts[0], 1: counts[1] + 1}), accepted`
Prompt exacto	Después de la función anterior también creé esta, pero al momento de realizar las pruebas, el resultado me da un número mayor a 1, lo cual es erróneo ya que se trata de una probabilidad. No sé en qué parte del código tengo errores, porque seguí el pseudocódigo que sale en la imagen. Necesito que me digas en que parte esta el error para poder continuar

Sugerencias relevantes	"hay un error, y está en la última línea:
	
	python"
	
	Python
	`return normalize({0: counts[0], 1: counts[1] + 1}), accepted  # incorrecto`
	
	`return normalize(counts), accepted`
	`# correcto`
	
	Se explica que counts[1] + 1 suma un 1 de más al conteo, como si hubiera una muestra aceptada además que no existe. Eso sesga la estimación hacia el valor de 1. Se explicó también que esta animalia en muestras lo suficientemente grandes no representan una alteración significativa a los datos, pero al trabajar con muestras pequeñas, la diferencia es muy grande.

Aceptado / rechazado / adaptado	Se modificó esa línea de código para eliminar el caso donde se suma un 1 adicional, lo que afecta pruebas con grupos muestrales pequeños.
Cómo se verificó	al correr -p 3 -l bosque --case rara -a rejection -n 10000, los resultados volvieron a estar dentro del rango posible para una probabilidad, y tienen afinidad con entre los datos, por lo tanto se concluyó que la modificación del error sugerida por la ia fue acertada.

Función: action_value

Campo	Contenido

Versión inicial (sin IA)	total = 0 for successor, probability in mdp.transitions(state, action): reward = mdp.reward(state, action, successor) total += reward + probability * values[successor] return total
Prompt exacto	Soy la persona 4 y tengo unos errores en mi código que no he logrado identificar necesito que me ayudes con esto y me guies a corregirlo y poder identificarlos entendiendo en que me equivoque para poder solucionarlo
Sugerencias relevantes	Recorrer los sucesores y sus probabilidades mediante mdp.transitions y obtener la recompensa con mdp.reward y por último calcular cada aporte como probability * (reward + mdp.discount * values[successor]) al final devolver el total fuera del ciclo.
Aceptado / rechazado / adaptado	Se conservaron el acumulador y el recorrido de sucesores del borrador. Se corrigió la expresión para que la probabilidad multiplicara tanto la recompensa inmediata como el valor futuro. También se incorporó el descuento al valor futuro. Se aceptaron estas sugerencias.
Cómo se verificó	Se ejecutaron python3 main.py -p 4 -l control -t y python3 main.py -p 4 -l cuadricula -t. Q(inicio, E) fue 1 y las otras tres acciones tuvieron valor 0.86. En cuadricula, los valores aproximados fueron N: 0.373852, E: 0.272666, S: 0.292216 y W: 0.306671. Estas pruebas comprobaron la función integrada con value_iteration.

Función: value_iteration

Campo	Contenido
Versión inicial (sin IA)	delta = 0 for state in mdp.states: actions = mdp.actions(state)

	if not actions:
	  continue
	
	best_value = 0
	
	for action in actions:
	  q = action_value(mdp, state, action, values)
	
	  if q > best_value:
	  best_value = q
	
	old_value = values[state]
	values[state] = best_value
	
	delta = abs(best_value - old_value)
	
	converged = delta <= tolerance

Prompt exacto	Soy la persona 4 y tengo unos errores en mi código que no he logrado identificar necesito que me ayudes con esto y me guies a corregirlo y poder identificarlos entendiendo en que me equivoque para poder solucionarlo
Sugerencias relevantes	Separar los valores anteriores de los nuevos para realizar barridos sincrónicos seleccionar el máximo valor entre las acciones disponibles y mantener los terminales en cero
Aceptado / rechazado / adaptado	Se sustituyó la actualización directa de values[state] por un diccionario new_values que reemplaza a values al terminar el barrido. Se reemplazó best_value = 0 por el máximo entre los valores de las acciones, para admitir resultados negativos. Se corrigió delta para conservar el mayor cambio de todos los estados en lugar del último y por ultimo se asignó cero explícitamente a los terminales y se cambió <= por <
Cómo se verificó	En control, la ejecución convergió en 2 barridos con V(inicio) = 1 y valor terminal cero. En cuadricula, convergió en 26 barridos con V(inicio) ≈ 0.373852 y ambos terminales en cero. El cambio máximo del barrido 25 fue 1.57695 × 10^-8todavía superior a la tolerancia; en el barrido 26 fue 6.91094 × 10^-9 inferior a 10^-8

Función: extract_policy

Campo	Contenido

Versión inicial (sin IA)	`def extract_policy(mdp, values):`
	`  policy = {}`
	`  for state in mdp.states:`
	`  best_action = None`
	`  best_value = float('-inf')`
	`  for action in mdp.actions(state):`
	`  q = action_value(mdp, state, action, values)`
	`  if q > best_value:`
	`  best_value = q`
	`  best_action = action`
	`  return policy`

Prompt exacto	"Claude, estoy haciendo el punto 5 de un taller sobre mdps y escribí esta función. Puedes revisar si mi codigo está bien? Si encuentras algún error dime cómo corregir."
Sugerencias relevantes	"Estas son mis sugerencias al revisar tu versión inicial de `extract_policy`. 1. Guardar la mejor acción en `policy`. El código encuentra correctamente la mejor acción para cada estado, pero nunca la agrega al diccionario. Por eso, la función siempre retorna `{}`. Debes agregar `policy[state] = best_action` después del ciclo que recorre las acciones, pero dentro del ciclo de estados. 2. Mantener la comparación estricta `>`. Esto ya está bien implementado. Permite conservar la primera acción cuando hay un empate, respetando el orden N, E, S, W. 3. Conservar `best_action = None`. También está correcto, porque los estados terminales no tienen acciones disponibles y deben quedar asociados a `None`."
Aceptado / rechazado / adaptado	Agregué la línea y corregí el error que hacía que la función retornara un diccionario vacío.
Cómo se verificó	Revisé que ahora la función dejara de retornar un diccionario vacío.